

ELEC520

Python programming

Spring 2026

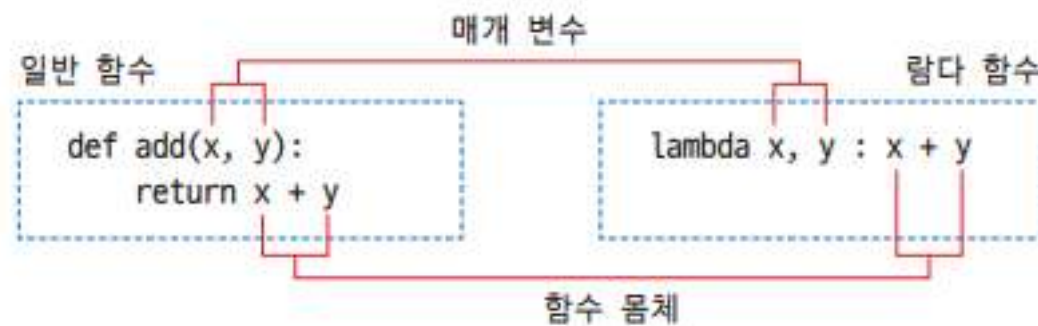
lambda function
Exception handling
File I/O

https://colab.research.google.com/drive/11X94uRWnODKmPrkM5r96MdKGu_MKwUO1?usp=sharing

lambda function

람다 함수 **lambda function**

- 이름 없는 함수, 익명 함수 **anonymous function**
- 1회용의 간단한 함수



[그림 10-1] 일반 함수와 람다 함수: 일반 함수는 def 키워드와 함수명, 매개변수, 콜론, 그리고 몸체로 구성되어 있으나 람다 함수는 매개변수와 함수의 기능만을 가진다.

코드 10-2 : 람다 함수를 정의하고 add라는 변수를 통해 참조하기

lambda_add.py

람다 함수를 이용한 덧셈

add = **lambda** x, y: x + y

print('100과 200의 합 :', add(100, 200))

실행결과

100과 200의 합 : 300

코드 10-3 : 인라인 람다 함수와 인자를 이용한 덧셈

inline_lambda_add.py

print('100과 200의 합 :', (**lambda** x, y: x + y)(100, 200))

위와 같이 별도의 라인으로 람다함수를 정의하지 않고
호출문 내에 람다 함수의 기능을 넣을 수 있다
이를 인라인 람다 함수라 한다

```
persons = [('GilDong', 'Hong', 27), ('SunSin', 'Lee', 46), ('YuSin', 'Gim', 34)]
```

1) 'Gim' 'Hong' 'Lee'의 성을 기준으로 정렬하여라. 정렬결과는 다음과 같다.

실행 결과

```
[('YuSin', 'Gim', 34), ('GilDong', 'Hong', 27), ('SunSin', 'Lee', 46)]
```

2) 나이가 많은 순서대로 내림차순 정렬하여라. 정렬결과는 다음과 같다.

실행 결과

```
[('SunSin', 'Lee', 46), ('YuSin', 'Gim', 34), ('GilDong', 'Hong', 27)]
```

다음은 문자열 10개가 담긴 리스트 `lst`이다. 문자열에 포함된 숫자의 **각 자리수 합**이 가장 큰 문자열을 출력하시오.

예를 들어, `'alpha12beta34'`에는 숫자 1, 2, 3, 4가 포함되어 있으며, 이들의 합은 $1 + 2 + 3 + 4 = 10$ 이다.

```
lst = ['alpha12beta34', 'gamma', 'delta56epsilon7', 'zeta', 'theta89', 'iota3kappa2lambda', 'mu', 'nu45xi', 'omicronpi', 'rho77sigma1']
```

실행 결과

문자열에 포함된 수들의 합이 가장 큰 문자열: delta56epsilon7

sorted()

`sorted(iterable, /, *, key=None, reverse=False)`

Return a new list containing all items from the iterable in ascending order.

A custom key function can be supplied to customize the sort order, and the reverse flag can be set to request the result in descending order.

- 특정 데이터를 기준으로 정렬하는 경우
 - key 매개변수 사용, 인자는 함수이름

```
>>> lst = ['hello', 'python', 'abcdefg', 'java']
>>> sorted(lst)
['abcdefg', 'hello', 'java', 'python']
>>> sorted(lst, key = len)
['java', 'hello', 'python', 'abcdefg']
>>> sorted(lst, key = lambda x: x[-1])
['java', 'abcdefg', 'python', 'hello']
```

파일

파일 열기와 모드

`open()`

파일 쓰기

`print()`

파일 읽기

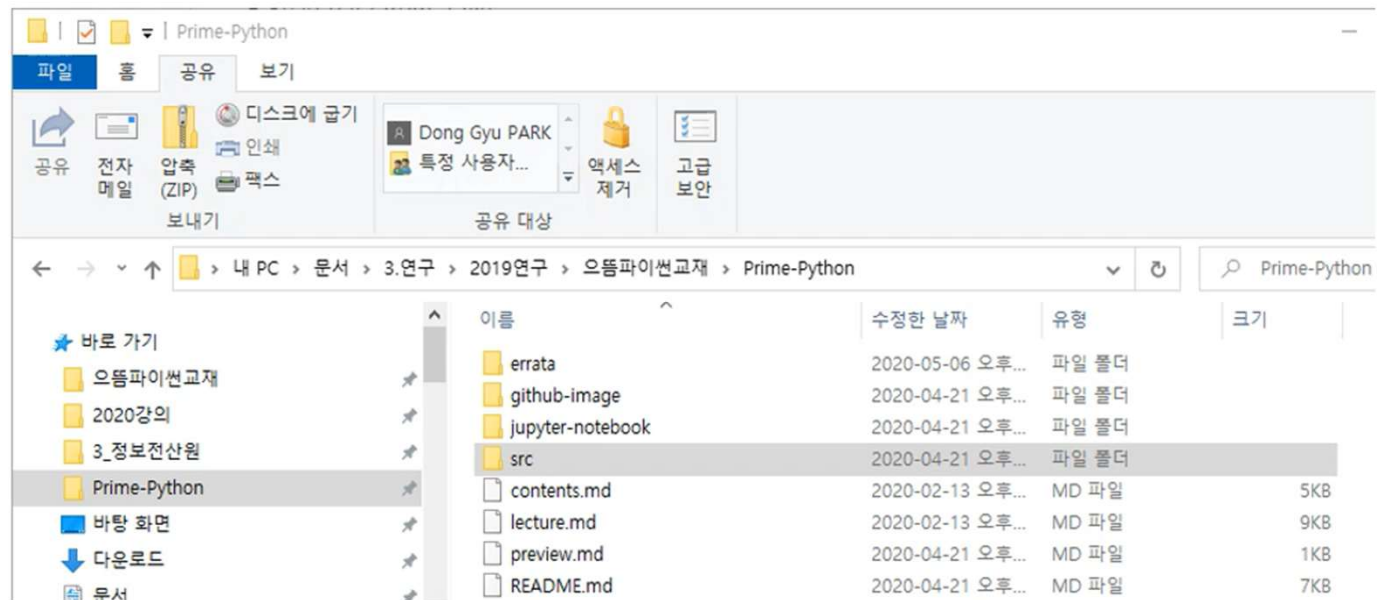
`for` 문 사용, `next()`

파일 닫기

`close()`, `with` 문 사용

파일file

- 컴퓨터의 저장 장치 내에 데이터를 저장하기 위해 사용하는 논리적인 단위
- **하드 디스크** **hard disk**나 **외장 디스크** **external disk**와 같은 저장 장치에 저장한 후 필요할 때 다시 불러서 사용하는 것이 가능



파이썬에서 파일을 사용하기 위한 절차

1. 파일 열기

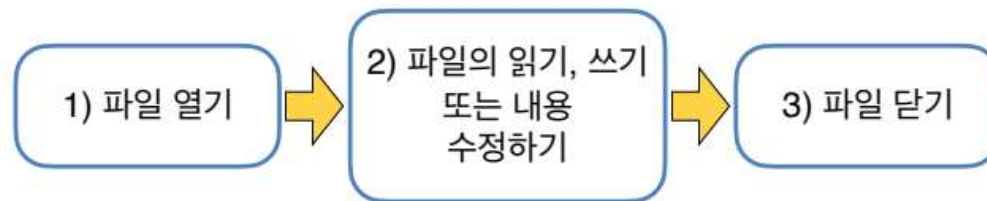
- 저장 장치 내의 위치와 파일 이름을 지정해서 파일을 가져온다.
- 컴퓨터 저장장치 내의 파일 위치는 **경로** `path`라고 함

2. 사용 목적에 따라 사용

- 내용 확인 및 새로운 내용 추가 또는 기존 내용 삭제 가능

3. 파일 닫기

- 파일을 모두 사용하고 나면 이 메모리를 시스템에서 사용할 수 있도록 반환



[그림 9-2] 파일을 열고 사용한 후 닫는 3단계의 절차

open()

- 파일 이름과 파일 오픈 모드의 인자를 가질 수 있음
- encoding이라는 인자를 통해 파일의 인코딩 형식 지정 가능
- 지정된 경로가 없을 경우 디폴트 경로는 파이썬 스크립트 파일이 있는 곳이 된다.

Syntax: 함수 정의

형식 파일객체 = `open`(파일이름, 파일모드)
 파일객체.close()

예 `infile = open("input.txt", "r")`
 `...`
 `infile.close()`

파일 객체

파일의 이름(name)

파일을 여는 모드(mode)

mode: read/write

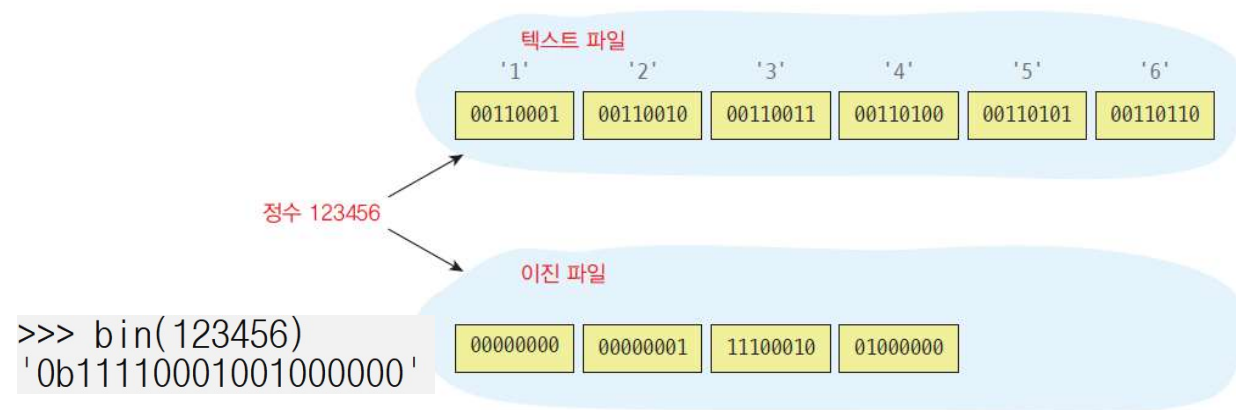
[표 9-3] 파일 오픈을 위한 여러 가지 모드와 그 의미

모드	의미
r	Read의 약자로 파일을 읽기 전용 모드로 연다. 이 모드는 파일 열기의 기본 모드이다. - 파일이 없을 경우 에러가 발생한다. - 읽기 전용 모드이므로 파일을 쓰려고 하면 오류가 발생한다.
w	Write의 약자로 파일을 쓰기 전용 모드로 연다. - 파일이 없을 경우 파일을 생성하며, 파일이 있을 경우 덮어쓰게 된다.
a	Append의 약자로 파일을 쓰기 모드로 열어서 기존 파일의 뒤에 새로 작성된 내용을 추가한다. - 기존 파일이 없을 경우 새로 파일을 만들어서 추가한다.
x	이 모드는 쓰기 전용으로 파일을 여는데, 파일을 새로 만들기 위해서 사용한다. - 이 모드는 a나 w와 달리 파일이 있을 경우 에러가 발생한다.
+	+ 기호는 읽기 쓰기 모드로 'r+'나 'w+', 'r+t'와 같은 조합으로 사용할 수 있다. 'r+'와 같이 표기하면 읽기 모드로 열어서 쓰기까지 가능하다. 반면, 'w+'와 같이 표기하면 쓰기 모드로 열어서 읽기까지 가능하다.

mode: text/binary

[표 9-4] 텍스트 파일과 이진 파일을 위한 모드의 종류

파일 모드	의미
t	Text의 약자로 텍스트 파일 형식으로 파일을 열거나 생성한다. 이 모드는 파일 열기의 기본 모드이다.
b	Binary의 약자로 이진 파일 형식으로 파일을 열거나 생성한다.



open()과 mode

```
f = open('hello.txt', 'w') # 파일 열기
```

파일을 쓰기모드(w)로 읽어서 파일 객체를 반환 : write() 메소드 호출하여 지정된 파일에 텍스트를 쓸 수 있음

```
f = open('hello.txt', 'wt') # 파일을 텍스트로 열기
```

위의 코드와 동일함 텍스트(t) 모드로 파일을 열어서 쓸 수 있음

```
f = open('hello.txt', 'wb') # 파일을 바이너리로 열기
```

wb모드는 바이너리 모드로 파일 쓰기 가능

```
f = open('hello.txt', 'r+t') # 텍스트 읽기 쓰기 모드
```

기본모드는 읽기 모드이지만 쓰기도 가능, 파일 없으면 오류 발생

```
f = open('hello.txt', 'w+t') # 텍스트 쓰기 읽기 모드
```

기본모드는 쓰기 모드이지만 읽기도 가능함

```
f = open('hello.txt', 'a+t') # 텍스트 추가 모드
```

파일의 모든 내용을 남겨두고 맨 뒤에 추가함. 읽기도 가능하며 파일이 없을 경우 만든다.

```
f.write('Hello World!') # hello.txt 파일에 이어쓰기
```

```
f.write('Love of my life, you've hurt me\n')
```

```
f.write('You've broken my heart and now you leave me\n')
```

\n을 입력하여야 파일 내 줄바꿈이 가능함. ' .. ' 내부에 따옴표를 출력하려면 \\'를 사용

파일 닫기

#버전 1

```
f = open('test.txt', 'w')    #파일을 연다.  
#여러 가지 작업을 한다.  
f.close()                    #파일을 닫는다.
```

#버전 2

```
with open('test.txt', 'w') as f:  
    f.write('김영희\n')  
    f.write('최자영\n')    #여러 가지 작업을 한다.  
  
#블록을 빠져나오면 자동으로 파일이 닫힘.
```



권장

파일에 쓰기 - print()

- print() 함수의 file 매개변수 활용

```
print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
        string inserted between values, default a space.
    end
        string appended after the last value, default a newline.
    file
        a file-like object (stream); defaults to the current sys.stdout.
    flush
        whether to forcibly flush the stream.
```

파일 내용 읽기

- for 문 사용 - 파일 내용 모두 읽기

```
with open('we_will_rock.txt') as f:
    for x in f:
        print(x.rstrip())
```

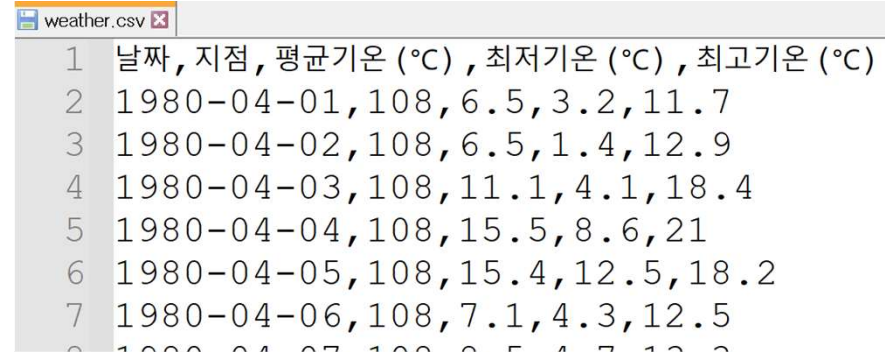
- 내장 함수 next() 사용하여 한 라인 읽기

```
with open('we_will_rock.txt') as f:
    print(next(f).rstrip()) #파일의 첫번째 라인
    print(next(f).rstrip()) #파일의 두번째 라인
    print(next(f).rstrip()) #파일의 세번째 라인
```


CSV 파일 처리

- Comma Separated Value
- CSV는 테이블 형식의 데이터를 저장하고 이동하는데 사용되는 구조화된 텍스트 파일
- CSV는 Microsoft Excel와 같은 스프레드시트에 적합
- 공공기관에서 제공하는 데이터는 CSV 형식의 데이터 세트가 많음
- CSV 모듈의 reader() 함수를 통해 comma 제거된 데이터에 접근 가능

```
import csv #comma seperated values
with open('weather.csv') as f:
    next(f) #목차 제거
    parsed_data = csv.reader(f)
```



날짜	지점	평균기온 (°C)	최저기온 (°C)	최고기온 (°C)
1980-04-01	108	6.5	3.2	11.7
1980-04-02	108	6.5	1.4	12.9
1980-04-03	108	11.1	4.1	18.4
1980-04-04	108	15.5	8.6	21
1980-04-05	108	15.4	12.5	18.2
1980-04-06	108	7.1	4.3	12.5

CSV 실습

```
import csv

with open('weather.csv') as f:
    next(f) #목차 제거
    parsed = csv.reader(f)
    print(next(parsed))
```

['1980-04-01', '108', '6.5', '3.2', '11.7']

- 평균 기온이 가장 높은 날은?
- 최저 기온이 가장 낮은 날은?

weather.csv
파일 다운 받기

	A	B	C	D	E
1	날짜	지점	평균기온(°C)	최저기온(°C)	최고기온(°C)
2	1980-04-01	108	6.5	3.2	11.7
3	1980-04-02	108	6.5	1.4	12.9
4	1980-04-03	108	11.1	4.1	18.4
5	1980-04-04	108	15.5	8.6	21
6	1980-04-05	108	15.4	12.5	18.2
7	1980-04-06	108	7.1	4.3	12.5
8	1980-04-07	108	8.5	4.7	13.3
9	1980-04-08	108	10.8	8.4	15.2
10	1980-04-09	108	10.3	5.7	16.1
11	1980-04-10	108	10.1	4.0	17.2

```
>>> 2/0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

예외 처리

try-except 문

try-except-else

try-except-finally

with 문

예외 Exception

- 사용자들은 잘못된 데이터를 입력할 수도 있고, 우리가 오픈하고자 하는 파일이 컴퓨터에 존재하지 않을 수도 있으며 인터넷이 다운될 수도 있다.



NOTE: 구문 오류와 예외

위에서 살펴본 바와 같이 문자열에서 닫는 따옴표를 빼먹는다면, if 문의 비교 후 콜론을 빼먹는 등의 파이썬 오류는 정해진 문법을 지키지 않았기 때문에 발생하므로 구문 오류, 문법 오류 혹은 **파싱 오류** `parsing error`라고 한다. 그러나 문법적으로는 올바른 프로그램이 수행 도중 예상하지 못한 입력 값이 들어오거나 비정상적 상황으로 인해 종료되는 경우도 있을 수 있다. 이러한 경우의 오류를 **예외** `exception`라고 지칭한다.

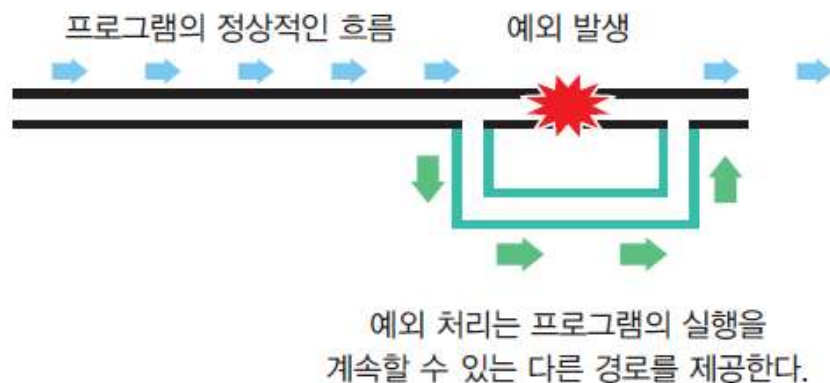


NOTE: 오류의 사전 탐지와 다루는 방법

프로그램에서 발생하는 오류에 대응하는 방법은 크게 두 가지 방향이 있다. 하나는 오류의 **사전 탐지 detection**이며, 다른 하나는 오류를 **적절하게 다루는 방법 handling**이다. 예를 들어서 파일을 읽어들이는 `open()` 함수를 고려해 보자. 이 함수에서 존재하지 않는 파일을 읽으려고 하면 오류가 발생하는데, 이를 방지하기 위해서 이 파일이 존재하는가를 미리 검사하는 사전 탐지 기법을 사용할 수 있다. 또 다른 경우는 파일이 존재하는 것을 확인하고 읽어 들이는 과정에서 이 파일의 형식이 잘못되거나 속성이 쓰기금지로 되어서 오류가 발생하는 불가피한 오류가 있다. 파이썬의 예외 처리는 이와 같이 예상하지 못한 오류에 대한 적절한 대응 방법을 사용자가 작성할 수 있는 기법이다.

예외 처리 Exception handling

- 오류가 발생했을 때 오류를 사용자에게 알려주고 모든 데이터를 저장하게 한 후에 사용자가 **우아하게(gracefully)** 프로그램을 종료할 수 있도록 하는 것이 바람직



a/b를 출력합니다. a, b를 입력해주세요.

a: 5

b: 0

Traceback (most recent call last):

File "C:\Users\Wheyjk\OneDrive\바탕 화면\WELC520\강의자료\파일\week09.py", line 116, in <module>

excption_tst2()

File "C:\Users\Wheyjk\OneDrive\바탕 화면\WELC520\강의자료\파일\week09.py", line 105, in excption_tst2

print(a/b)

ZeroDivisionError: division by zero



a/b를 출력합니다. a, b를 입력해주세요.

a: 5

b: 0

0으로는 나눌 수 없어요. b를 0아닌 수로 입력해주세요.

a: 5

b: 2

2.5

정수: 구

Traceback (most recent call last):

File "C:\Users\Wheyjk\OneDrive\바탕 화면\WELC520\강의자료\파일\week09.py", line 94, in <module>

excption_tst1()

File "C:\Users\Wheyjk\OneDrive\바탕 화면\WELC520\강의자료\파일\week09.py", line 91, in excption_tst1

n = int(input("정수: "))

ValueError: invalid literal for int() with base 10: '구'



정수: 구

정수를 입력해주세요.

정수: 9.2

정수를 입력해주세요.

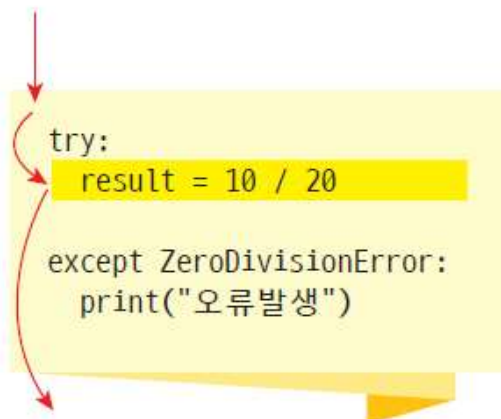
정수: 3

정수 입력 성공!

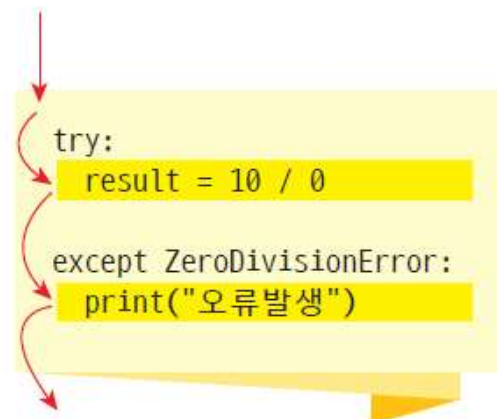
Syntax: 예외 처리

형식 `try:`
 예외가 발생할 수 있는 문장
`except(오류):`
 예외를 처리하는 문장

예 `try:`
 `z = x/y` 예외가 발생할 수 있는 문장
`except ZeroDivisionError:` 예외
 `print ("0으로 나누는 예외")`



예외가 발생하지 않은 경우



예외가 발생한 경우

try-except-else

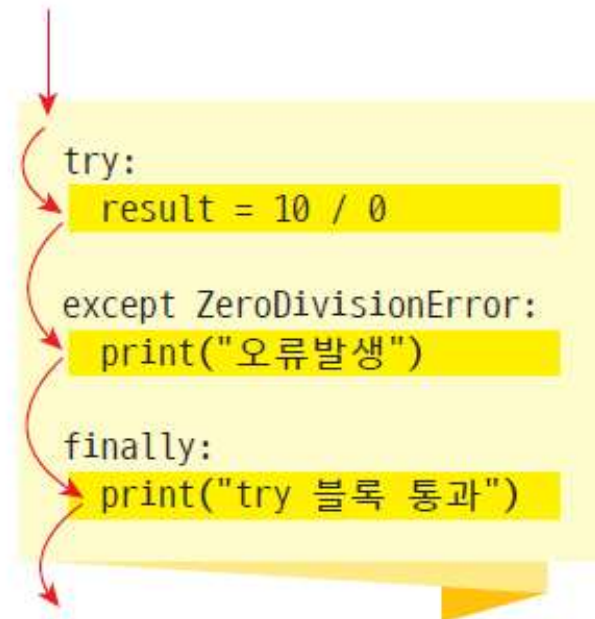
```
def test_ValueError():  
    while True:  
        try:  
            n = int(input('정수: '))  
            print('정수 입력 성공!')  
            break  
        except ValueError:  
            print('정수를 입력해주세요.')
```

```
def test_ValueError_ver_else():  
    while True:  
        try:  
            n = int(input('정수: '))  
        except ValueError:  
            print('정수를 입력해주세요.')        else:  
            print('정수 입력 성공!')  
            break
```

try-except-finally



예외가 발생하지 않은 경우



예외가 발생한 경우

next() 사용해서 파일 내용 모두 읽기

```
with open('we_will_rock.txt') as f:

    while True:
        try:
            print(next(f).rstrip())
        except StopIteration:
            break
```